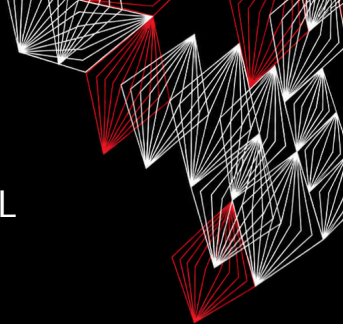


TEMA 101: ARQUITECTURA DEL SISTEMA

Erick Ricardo Martinez Martinez
Luis Fernando Torres Hernández

Laboratorio de Supercómputo y
Visualización en Paralelo

Junio 03, 2026



Contenido

- 1 101.1 Determinar y configurar ajustes de hardware
- 2 101.2 Arranque del sistema
- 3 101.3 Cambiar niveles de ejecución / objetivos y apagar/reiniciar

Introducción a la configuración de hardware

Objetivo

Aprender a determinar, inspeccionar y configurar los dispositivos de hardware en un sistema Linux.

- ▶ Activación y detección de dispositivos.
- ▶ Inspección de hardware mediante comandos.
- ▶ Archivos de información y dispositivos.
- ▶ Gestión de almacenamiento.

Inspección de dispositivos en Linux

Comandos clave

- ▶ `lspci`: Lista dispositivos PCI.
- ▶ `lsusb`: Lista dispositivos USB.
- ▶ `lsblk`: Muestra dispositivos de bloque (discos).
- ▶ `lsmod`: Muestra todos los módulos cargados actualmente.
- ▶ `dmesg`: Mensajes del kernel durante el arranque.

Interpretando la salida de lsusb

Ejemplo

```
$ lsusb
```

```
Bus 001 Device 002: ID 8087:0026 Intel Corp.
```

```
Bus 001 Device 003: ID 046d:c52b Logitech USB Receiver
```

Campos importantes

- ▶ **Bus:** Número del bus USB al que está conectado el dispositivo.
- ▶ **Device:** Identificador asignado por el kernel.
- ▶ **ID:** Identificador hexadecimal del fabricante y dispositivo.
- ▶ **Descripción:** Nombre reconocido del dispositivo.

Interpretando la salida de lspci

Ejemplo

```
$ lspci
```

```
00:02.0 VGA compatible controller:
```

```
Intel Corporation UHD Graphics
```

```
00:14.0 USB controller:
```

```
Intel Corporation USB 3.0 xHCI Controller
```

Campos importantes

- ▶ **00:02.0**: Bus, dispositivo y función PCI.
- ▶ **VGA compatible controller**: Clase del dispositivo.
- ▶ **Intel Corporation**: Fabricante.
- ▶ **UHD Graphics**: Modelo detectado.

Interpretando la salida de lsmod

Ejemplo

```
$ lsmod
```

Module	Size	Used by
nvidia	62259200	35
iwlwifi	393216	1
usb_storage	81920	1

Columnas

Module Nombre del módulo cargado.

Size Tamaño del módulo en memoria (bytes).

Used by Número de módulos o procesos que lo utilizan.

Administración de módulos: modprobe

¿Qué hace?

Carga o descarga módulos del kernel junto con sus dependencias.

Cargar un módulo

```
$ sudo modprobe usb_storage
```

Eliminar un módulo

```
$ sudo modprobe -r usb_storage
```

Importante

A diferencia de `insmod`, `modprobe` resuelve automáticamente las dependencias necesarias.

Consultando información: modinfo

Ejemplo

```
$ modinfo iwlmwifi
```

Información mostrada

- ▶ Nombre del módulo.
- ▶ Autor.
- ▶ Descripción.
- ▶ Versión.
- ▶ Licencia.
- ▶ Dependencias.
- ▶ Alias de dispositivos compatibles.
- ▶ Ubicación del archivo .ko.

Archivos de dispositivo y /sys

Directorios importantes

- ▶ /dev: Archivos de dispositivo (ej. /dev/sda).
- ▶ /sys: Información de dispositivos y kernel.
- ▶ /proc: Información de procesos y sistema.

Tipos de dispositivos

- ▶ **De bloque:** Discos, USB (lectura/escritura por bloques).
- ▶ **De carácter:** Teclado, mouse (flujo de bytes).

Dispositivos de almacenamiento

Nomenclatura tradicional vs. moderna

- ▶ `/dev/hd*`: Discos IDE (antiguo).
- ▶ `/dev/sd*`: Discos SATA, SCSI, USB (actual).
- ▶ `/dev/nvme0n1`: Discos NVMe.

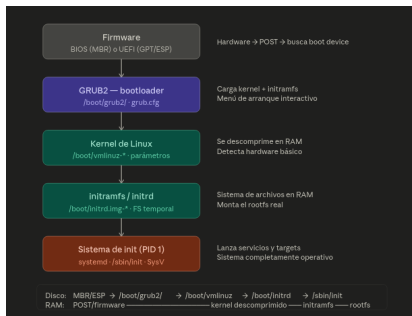
Particiones

`/dev/sda1`, `/dev/sda2` (primera partición, segunda partición).

Introducción al arranque

Secuencia de arranque en Linux

- ❶ BIOS o UEFI (Power-On Self Test).
- ❷ Cargador de arranque (GRUB, LILO).
- ❸ Kernel de Linux.
- ❹ Init / systemd (primer proceso).



BIOS vs UEFI

BIOS

- ▶ Firmware tradicional.
- ▶ MBR (Master Boot Record).
- ▶ Limitación de 2TB en discos.

UEFI

- ▶ Moderno, interfaz gráfica.
- ▶ Tabla de particiones GPT.
- ▶ Arranque seguro (Secure Boot).

El Cargador de Arranque: GRUB

Funciones principales

- ▶ Permite seleccionar el sistema operativo a iniciar.
- ▶ Permite elegir entre múltiples kernels instalados.
- ▶ Permite modificar temporalmente los parámetros de arranque.
- ▶ Carga el kernel y el sistema de archivos inicial (initramfs).

Parámetros del kernel

Los parámetros de arranque suelen tener la forma:

opcion=valor

y se agregan a la línea del kernel desde el menú de GRUB (presionando e durante el arranque).

Ejemplo

```
linux /vmlinuz ... root=/dev/sda2 rw init=/bin/bash
```

Parámetros comunes del kernel

Parámetro	Descripción
root=	Raíz que será montada durante el arranque.
rootflags=	Opciones adicionales para montar el sistema de archivos raíz.
ro	Monta inicialmente la raíz en modo sólo lectura.
quiet	Reduce los mensajes durante el arranque.
systemd.unit=	Inicia una unidad específica de systemd.
init=	Especifica el programa que actuará como proceso PID 1.
acpi=	Controla el uso de ACPI para gestión de energía y hardware.
mem=	Limita la memoria RAM visible para el kernel.
maxcpus=	Limita el número de CPUs utilizadas durante el arranque.
vga=	Configura el modo gráfico de consola (obsoleto en muchos sistemas).
rq	Habilita las funciones Magic SysRq para depuración.

Configuración permanente de parámetros

Archivo de configuración

En sistemas basados en GRUB, los parámetros permanentes se definen en:

`/etc/default/grub`

Variable principal

`GRUB_CMDLINE_LINUX="quiet splash"`

Inicialización del sistema: systemd

Definición

Un **demonio (daemon)** es un programa que se ejecuta en segundo plano y proporciona servicios al sistema operativo o a otros programas.

- ▶ No requiere interacción directa del usuario.
- ▶ Generalmente inicia durante el arranque del sistema.
- ▶ Permanece ejecutándose mientras el sistema está activo.
- ▶ Suele finalizar su nombre con la letra d.

Ejemplos

- ▶ sshd: Acceso remoto mediante SSH.
- ▶ cron: Ejecución programada de tareas.
- ▶ rsyslogd: Registro de eventos del sistema.
- ▶ httpd: Servidor web Apache.

Proceso de arranque del sistema Linux

- ❶ El gestor de arranque (GRUB) carga el núcleo (*kernel*) en memoria RAM.
- ❷ El kernel toma el control de la CPU e inicializa los componentes fundamentales del sistema:
 - ▶ Gestión de memoria.
 - ▶ Detección de hardware.
 - ▶ Controladores de dispositivos.
- ❸ El Kernel carga el **initramfs** (Initial RAM Filesystem), un sistema de archivos temporal que contiene los módulos necesarios para acceder al sistema de archivos raíz real.
- ❹ Una vez disponible la partición raíz, el núcleo monta los sistemas de archivos definidos en: **/etc/fstab**
- ❺ Finalmente se ejecuta el proceso `init`, que se convierte en el primer proceso del espacio de usuario (PID 1).

Sistemas de inicialización (init)

¿Qué es init?

Es el primer proceso ejecutado por el kernel (PID 1) y es responsable de iniciar servicios, demonios y preparar el sistema para los usuarios.

- SysV init** Basado en scripts ubicados en `/etc/init.d/`. Ejecuta servicios secuencialmente y utiliza niveles de ejecución (runlevels).
- Upstart** Sistema orientado a eventos desarrollado por Ubuntu. Mejoró el tiempo de arranque y la gestión dinámica de servicios.
- systemd** Sistema de inicialización moderno utilizado por la mayoría de las distribuciones actuales. Inicia servicios en paralelo, gestiona dependencias y utiliza unidades (*units*).

Comandos comunes de systemd

Administración de servicios

```
systemctl status sshd
```

Muestra el estado actual del servicio SSH.

```
systemctl start sshd
```

Inicia el servicio SSH.

```
systemctl stop sshd
```

Detiene el servicio SSH.

```
systemctl restart sshd
```

Reinicia el servicio SSH.

Inspección de inicialización

Herramienta	Uso principal
<code>dmesg</code>	Verificar detección de hardware, módulos y errores del kernel.
<code>journalctl -b</code>	Consultar los eventos ocurridos durante el último arranque.
<code>systemctl status</code>	Verificar el estado de un servicio específico.
<code>systemd-analyze</code>	Medir y analizar el tiempo de arranque.
<code>systemd-analyze blame</code>	Identificar los servicios que más tiempo consumen durante el inicio.

Introducción a los niveles de ejecución

SysVinit (tradicional)

Niveles del 0 al 6:

- ▶ 0: Apagado
- ▶ 1: Monousuario
- ▶ 3: Multiusuario sin GUI
- ▶ 5: Multiusuario con GUI
- ▶ 6: Reinicio

systemd (moderno)

- ▶ `poweroff.target`
- ▶ `rescue.target`
- ▶ `multi-user.target`
- ▶ `graphical.target`
- ▶ `reboot.target`

Trabajando con systemd targets

Consultar y modificar objetivos

- ▶ `systemctl get-default`
- ▶ `systemctl set-default multi-user.target`
- ▶ `systemctl set-default graphical.target`
- ▶ `systemctl isolate rescue.target`

Equivalencias

`init 1` -> `systemctl isolate rescue.target`

`init 3` -> `systemctl isolate multi-user.target`

`init 5` -> `systemctl isolate graphical.target`

Dependencias entre targets

Inspección de dependencias

```
systemctl list-dependencies
```

```
systemctl list-dependencies graphical.target
```

¿Para qué sirve?

Permite visualizar los servicios y objetivos que deben iniciarse para alcanzar un target determinado.

Consultar el estado actual

Comandos útiles

`runlevel`

`who -r`

`systemctl get-default`

Interpretación

Estos comandos permiten conocer el nivel de ejecución actual o el objetivo configurado por defecto en el sistema.

Apagado y reinicio del sistema

Comandos tradicionales

- ▶ `shutdown -h now`
- ▶ `shutdown -r +5`
- ▶ `shutdown -c`

Comandos modernos

- ▶ `systemctl poweroff`
- ▶ `systemctl reboot`
- ▶ `systemctl rescue`

Buenas prácticas

Utilice `shutdown` o `systemctl` para permitir que los servicios finalicen correctamente y evitar corrupción de datos.

Comparativa: SysVinit vs systemd

Acción	SysVinit	systemd
Apagar	init 0	systemctl poweroff
Reiniciar	init 6	systemctl reboot
Modo rescate	init 1	systemctl rescue
CLI multiusuario	init 3	systemctl isolate multi-user.target
GUI	init 5	systemctl isolate graphical.target

Resumen de la unidad

- ▶ **Hardware:** `lspci`, `lsusb`, `lsblk`, `lsmod`.
- ▶ **Arranque:** BIOS/UEFI, GRUB, Kernel, `initramfs` y `systemd`.
- ▶ **Inicialización:** demonios, servicios y PID 1.
- ▶ **Runlevels y targets:** equivalencias entre SysVinit y `systemd`.
- ▶ **Administración:** `systemctl`, `journalctl` y `systemd-analyze`.
- ▶ **Apagado y reinicio:** uso seguro de `shutdown` y `systemctl`.

¿Preguntas o comentarios?